Section 7: Reflection Prompts

Student: Xiaohong Yuan (`xyuan04@nyit.edu`) **Student ID:** 1360530 **Track:** Track C — Live Order Tracking (MVP)

1. Three Defects

3 SPECIFIC PROBLEMS IN AI OUTPUT

> *Describe three specific problems in AI output (wrong, risky, or wasteful). For each: what it was, how you found it, and how you fixed it.*

Defect 1: Scope Creep & Unrequested Heavy Infrastructure (Redis & Celery)

WASTEFUL / RISKY

What it was: During Slice 6 planning and architectural design (`design.md`) for real-time order tracking, the AI proposed introducing an external Redis Pub/Sub message broker and a Celery worker queue, adding two heavy third-party dependencies (`redis-py`, `celery`) and external daemon process orchestration.

How I found it: Architectural review of `design.md` against project constraints. MiniDash is specified as an educational, self-contained REST API designed for instant local execution without external background daemons.

How I fixed it: Rejected the external message broker entirely. Designed and implemented an in-memory, thread-safe pub/sub broadcaster class (`Broadcaster` in `app/services/broadcaster.py`) using Python's native `asyncio.Queue`, fulfilling Server-Sent Events (SSE) requirements with zero external infrastructure.

Defect 2: Schema Validation Interception Violating Domain Contract (HTTP 422 vs. HTTP 400)

WRONG / CONTRACT VIOLATION

What it was: In Slice 1 (`app/schemas/order.py`) and Slice 4, the AI typed `OrderStatusUpdate.status: OrderStatus` (a Pydantic enum). When an invalid status string (e.g., `"FLYING"`) was sent to `PATCH /orders/{id}/status`, Pydantic automatically rejected the request with HTTP `422 Unprocessable Entity` before it reached the service layer.

How I found it: Ran automated test verification against acceptance criterion **AC-2.9** (*"If the requested status is not a recognized OrderStatus enum value, MiniDash shall respond with HTTP 400 Bad Request"*). The test failed due to receiving HTTP 422 instead of HTTP 400.

How I fixed it: Changed `OrderStatusUpdate.status` to plain `str` in Pydantic to allow raw inputs to reach `OrderService.update_order_status`. In the service layer, explicit domain logic validates against `OrderStatus.__members__`, returning HTTP `400 Bad Request` for unrecognized statuses, and HTTP `409 Conflict` for invalid FSM transitions.

Defect 3: Non-Deterministic Ordering in Status History Queries

WRONG / SUBTLE FLAKINESS

What it was: In Slice 5 (`GET /orders/{id}/tracking`), the AI wrote the SQL query fetching tracking history as: `SELECT status, timestamp FROM order_status_history WHERE order_id = ? ORDER BY timestamp ASC`. Because automated tests and driver simulations advance states in sub-second intervals, sequential transitions shared identical timestamp strings in SQLite's ISO text format, resulting in sporadic out-of-order records.

How I found it: Observed flaky test assertions in rapid simulation runs where the returned history array occasionally placed `READY` before `PREPARING` when both shared identical timestamp seconds.

How I fixed it: Modified the query to `ORDER BY id ASC` (supported by index `idx_order_status_history_order_id`). Because `id` is an auto-incrementing primary key, it guarantees strictly monotonic chronological ordering across all transitions regardless of clock resolution.

2. Where AI Helped Most and Least

COMPARATIVE WORKFLOW ANALYSIS

> *Give one example of each and say why.*

▲ Helped Most: Test Scaffolding & Matrix Coverage

Example: Authoring the 47-test automated verification suite, particularly `tests/test_fsm.py` and `tests/test_eta.py`.

Why: The FSM lifecycle contains a transition matrix of 6 distinct states (36 state pairs), plus conditional cancellation rules, terminal state immutability, and 400 vs 404 vs 409 error branches. Hand-writing boilerplate pytest fixtures, mock SQLite connections, and parameterized assertions for every permutation is tedious and error-prone. The AI rapidly synthesized exhaustive test cases directly from EARS criteria, providing immediate automated regression safety across every slice.

▼ Helped Least: Async Concurrency & Stream Lifecycles

Example: Implementing the Server-Sent Events (SSE) streaming endpoint (`GET /orders/{id}/tracking/stream`) with client disconnect detection in Slice 6.

Why: The AI struggled with asynchronous generator lifecycles and Starlette streaming protocol subtleties. Its initial output used a blocking polling loop (`while True: await asyncio.sleep(1)`) that repeatedly queried SQLite rather than reacting to push events. It also failed to handle client disconnections, causing queue memory leaks. Human engineering was required to design the pub/sub listener pattern and guaranteed cleanup in `try...finally`.

3. Prompting Progression Across Slices

PROMPT ENGINEERING EVOLUTION

> *What did you change about how you prompted between your first and last slice?*

Between Slice 1 and Slice 8, my prompting transitioned from **open-ended conversational requests** to **strict contractual specifications**:

- **Early Slices (Slices 1 & 2):** Prompts were conversational (e.g., *"Implement the database schema and models for tracking"*). This led to AI hallucinating unnecessary libraries, introducing extraneous helper modules, and defaulting to standard Pydantic validation patterns that violated our required HTTP error code contracts.
- **Later Slices (Slices 6, 7 & 8):** Prompting evolved into structured contracts enforcing three guardrails:
  1. **Strict File & Scope Boundaries:** Explicitly constrained files to touch and banned external packages (e.g., *"Modify only `app/services/broadcaster.py`. Zero new dependencies in `requirements.txt`. Use standard library `urllib` for CLI"*).
  2. **EARS Acceptance Criteria Binding:** Direct quotation of criteria (e.g., *"Satisfy AC-4.1, AC-4.2, and AC-4.4: return 200 with `text/event-stream`, emit initial snapshot immediately, auto-close stream on terminal status"*).
  3. **Pre-emptive Failure Mode Constraints:** Instructing edge-case handling before generation (e.g., *"Wrap consumer in `try...finally` to call `broadcaster.unsubscribe` on disconnect"*).

Impact: Shifting to contract-first prompting eliminated scope creep and reduced iteration cycles from multiple manual correction rounds to single-shot passing code that satisfied all verification tests immediately.

4. A Decision You Did Not Delegate

ARCHITECTURAL & QUALITY OWNERSHIP

> *Name one design or quality decision you kept for yourself and explain why.*

The Decision: Transactional FSM Invariants & Database Rollback Strategy (Single-Source-of-Truth Lifecycle Integrity).

Explanation & Why I Kept It: The AI repeatedly proposed checking state transitions in Python application memory before issuing an independent `UPDATE orders` statement, followed by an independent `INSERT INTO order_status_history` query without explicit database transaction demarcation.

State machines in concurrent web applications cannot rely on naive, non-atomic application-level checks. If the update to `orders` succeeds but the subsequent insert to `order_status_history` fails (e.g., due to constraint violation, disk failure, or process interruption), the system enters an illegal, corrupted state where an order has transitioned without an audit record. I deliberately reserved and enforced this architectural boundary:

- Wrapped state validation, `orders` update, and `order_status_history` insertion inside an atomic SQLite transaction (`BEGIN IMMEDIATE ... COMMIT / ROLLBACK`).
- If any step fails, the entire transition is aborted cleanly, guaranteeing zero partial writes.
- Enforced terminal states (`DELIVERED`, `CANCELLED`) as immutable mathematical sinks at the service boundary.

Engineering Rationale: Architectural integrity, fault domains, and transactional boundaries are non-delegable engineering responsibilities. LLMs optimize for syntactic completion of the happy path, whereas robust software engineering demands defensive design against partial failure modes.

5. Limits of Your Evidence

VERIFICATION BOUNDARY & FUTURE HARDENING

> *What is still not verified in your feature, and how would you check it with more time?*

While our automated suite of 47 tests achieves 100% pass rate across functional requirements, three real-world operational aspects remain outside the scope of current automated evidence:

1. **Multi-Worker ASGI Concurrency:** The current SSE broadcaster relies on an in-process, in-memory `asyncio.Queue`. When deploying FastAPI with multiple worker processes (e.g., `uvicorn app.main:app --workers 4`), updates in Worker A will not trigger SSE pushes for client connections handled by Worker B.
   How to verify with more time: Set up a multi-worker Uvicorn test harness with a shared IPC/Redis pub/sub backend and run concurrent multi-client simulations to verify cross-worker event delivery.
2. **High-Throughput SQLite Write Contention (`SQLITE_BUSY`):** The automated test suite executes sequential or lightly concurrent requests. Under intense concurrent write traffic from dozens of simultaneous driver simulations, SQLite's single-writer architecture could experience lock contention.
   How to verify with more time: Execute a Locust or `wrk` load test simulating 500 concurrent drivers advancing order statuses simultaneously to measure SQLite lock wait latency, p99 response times, and verify zero unhandled database lock exceptions.
3. **Transient Network Disconnection & Event Replay:** If a client's mobile network drops for 5 seconds and reconnects, the current SSE implementation emits the latest snapshot upon reconnection, but does not provide an HTTP `Last-Event-ID` replay buffer to reconstruct intermediate transient status transitions that occurred during the outage window.
   How to verify with more time: Use Toxiproxy to inject random latency, packet drops, and socket termination into active SSE streams, asserting that clients recover with proper reconciliation.